# 《软件工程》

## 实验报告三：面向对象的系统建模A

| | | | |
|---|---|---|---|
| 姓　　名： | 房家和 | 学　　号： | **202410360203** |
| 院　　系： | 计算机与信息学院 | 专　　业： | 计算机科学与技术 |
| 实 验 室： | **J1307** | 实验日期： | **2026** 年 **10** 月 **16** 日 |
| 总评成绩： | | 审阅教师： | 覃兵文 |

# 一、实验目的

（1）掌握 UML 建模工具 Rational Rose 软件的安装和基本操作；

（2）掌握面向对象设计模型，包括类和类间的关系建模；

（3）能够构建类图静态模型。

# 二、实验环境

操作系统：Windows 11 家庭中文版；

编程语言：Java，JDK 21.0.8，在命令行中用 javac / java 编译运行；

建模工具：本机未安装 Rational Rose，类图按 UML 2 类图表示法绘制（由 Claude Code 编写的 Python 脚本生成 SVG 矢量图，配色参照 Rose）；

版本控制：Git 2.52.0（Git for Windows），代码托管平台 GitHub；

AI 编程助手：Claude Code（用于辅助编写代码、绘制类图）。

# 三、实验要求

（1）准确规范使用 UML 模型，包括类名，属性，方法，关系和多重性，角色，可见性；

（2）独立完成，禁止抄袭；

（3）使用 Rational Rose 创建模型；

（4）选择自己熟悉的语言实现之，代码提交到自己的 git 仓库，代码地址写到实验报告中；

（5）按照实验报告模板，编写实验报告，复制创建的模型贴到 Word 报告中；

（6）文件以“学号-姓名-软件工程实验三”的方式命名，提交到长江雨课堂。

# 四、实验内容

## 0. UML 类图表示法与代码组织

本实验用到的 UML 类图元素如表1 所示。除题目要求的操作外，各类图中省略了构造方法和简单的 get / set 方法。

表1 本实验用到的 UML 类图元素

| 元素 | 表示法 | 含义 |
|---|---|---|
| 类 | 三栏矩形：类名 / 属性 / 操作 | 抽象类的类名用斜体并标注 {abstract}；枚举加构造型 «enumeration» |
| 可见性 | + public，- private，# protected，~ package | 写在属性和操作前面 |
| 属性 | 可见性 名称 : 类型 [= 初值] | 静态成员加下划线 |
| 操作 | 可见性 名称(参数名 : 类型, …) : 返回类型 | |
| 关联 | 实线，线上可写关联名 | 两端标注多重性 |
| 聚合 | 实线，空心菱形在整体一端 | 部分可以独立于整体存在，也可以被多个整体共享 |

| | | |
|---|---|---|
| 组合 | 实线，实心菱形在整体一端 | 部分只属于一个整体，随整体创建和销毁 |
| 泛化 | 实线，空心三角形指向父类 | 继承关系 |
| 依赖 | 虚线，开放箭头指向被使用的类 | 一个类用到另一个类 |
| 多重性 | 1、0..1、0..*、1..*、1..3 等 | 这一端可以有多少个对象与另一端的一个对象关联 |
| 注释 / 约束 | 折角矩形，虚线连到被注释的元素；约束写在 { } 中 | |

四道题的代码放在仓库的 `lab3-oo-modeling` 目录下，每道题一个包（`a1_1` ~ `a1_4`），每个包中的 `Demo` 类演示模型的用法；类图的 SVG 文件放在 `diagrams` 目录下。编译和运行命令如下：

```
cd lab3-oo-modeling
# 编译全部源文件，输出到 out 目录
javac -encoding UTF-8 -d out $(find src -name "*.java")
# 运行某道题的演示程序，例如 A1-4
java -Dstdout.encoding=UTF-8 -cp out a1_4.Demo
```

## 1. A1-1 纯种马 Thoroughbred

**描述：**

用 Thoroughbred 类对优良品质的马进行建模。它有 3 个属性：mother、father、birthyear，还有 3 个操作：getCurrentAge()、getFather() 和 getMother()。每个属性都有名字、类型和可见性级别，所有属性都是私有的；用可见性级别、带名字和类型的参数以及返回类型来表示每个操作。

**建模分析：**

类名为 Thoroughbred。三个属性都是私有的（-）：mother 和 father 的类型就是 Thoroughbred 本身，因为纯种马的父母也是纯种马；birthyear 为 int。

三个操作都是公有的（+）：getCurrentAge(currentYear : int) : int 返回这匹马到 currentYear 年时的年龄，年份由调用者传入而不是在方法内部读取系统时间，便于测试，也满足“带名字和类型的参数”的要求；getFather() 和 getMother() 的返回类型都是 Thoroughbred。

mother 和 father 也可以画成 Thoroughbred 到自身的关联（角色名 mother、father，多重性 0..1）。题目明确要求把它们作为属性，所以放在属性栏中；父母未知时取 null，对应多重性中的 0。

**类图：**

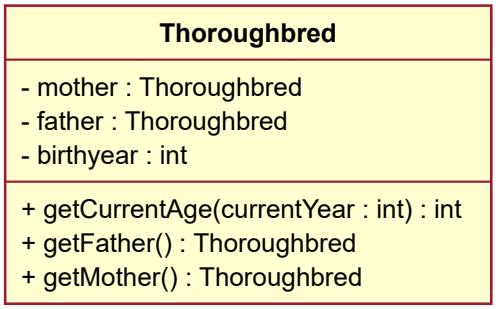



图1 A1-1 Thoroughbred 类图

**代码：**

Thoroughbred.java：

```
package a1_1;

/** 优良品种的马（纯种马），记录父母和出生年份。 */
public class Thoroughbred {
    private Thoroughbred mother;
    private Thoroughbred father;
    private int birthyear;

    /** 血统未知的马，父母传 null。 */
    public Thoroughbred(Thoroughbred mother, Thoroughbred father, int birthyear) {
        this.mother = mother;
        this.father = father;
        this.birthyear = birthyear;
    }

    /** 返回这匹马在 currentYear 年的年龄（周岁）。 */
    public int getCurrentAge(int currentYear) {
        if (currentYear < birthyear) {
            throw new IllegalArgumentException("currentYear 不能早于出生年份 " + birthyear);
        }
        return currentYear - birthyear;
    }

    public Thoroughbred getFather() {
        return father;
    }

    public Thoroughbred getMother() {
        return mother;
    }
}
```

**运行结果：**

演示程序创建父亲（2015 年出生）、母亲（2017 年出生）和它们 2023 年出生的小马，结果如图2所示：小马在 2026 年为 3 岁，getFather() / getMother() 返回的正是创建时传入的父母对象；年份早于出生年份时抛出异常。

```
$ java -Dstdout.encoding=UTF-8 -cp out a1_1.Demo
=== A1-1 Thoroughbred ===
foal 在 2026 年的年龄：3
foal.getFather() == sire：true
foal.getMother() == dam：true
父亲在 2026 年的年龄：11
母亲在 2026 年的年龄：9
sire 的父亲（血统未知）：null
getCurrentAge(2020) 抛出异常：currentYear 不能早于出生年份 2023
```

图2 A1-1 运行结果

## 2. A1-2 学院 College、建筑 Building 与课程 Course

**描述：**

College 有一个包含 Building 对象的聚合，这表示建筑构成了学院。学院也有一个包含课程的组合。Course 类也受到约束性注释（注释 Course 一定在某个 Building 中进行）。

**建模分析：**

三个类之间的关系如表2 所示。

表2 A1-2 类之间的关系

| 关系 | 类型 | 多重性 | 说明 |
|---|---|---|---|
| College — Building | 聚合（空心菱形在 College 端） | College 1<br>Building 1..* | 建筑构成学院，但建筑可以独立存在：Building 由外部创建，再通过 `addBuilding` 加入学院 |
| College — Course | 组合（实心菱形在 College 端） | College 1<br>Course 0..* | 课程属于学院，由学院创建和管理：Course 的构造方法是包内可见的，只能通过 `College.openCourse` 创建。刚成立的学院可能还没有开课，所以取 0..* |
| Course — Building | 关联“在…中进行” | Course 0..*<br>Building 1 | 每门课程在 1 个建筑中进行，一个建筑中可以安排多门课程 |

约束 {Course 一定在某个 Building 中进行} 用注释框连到 Course 上。代码中有两处保证这条约束：Course 的构造方法用 `Objects.requireNonNull` 拒绝空的 building；`College.openCourse` 在创建课程前检查 building 不为空，并且属于本学院。

**类图：**

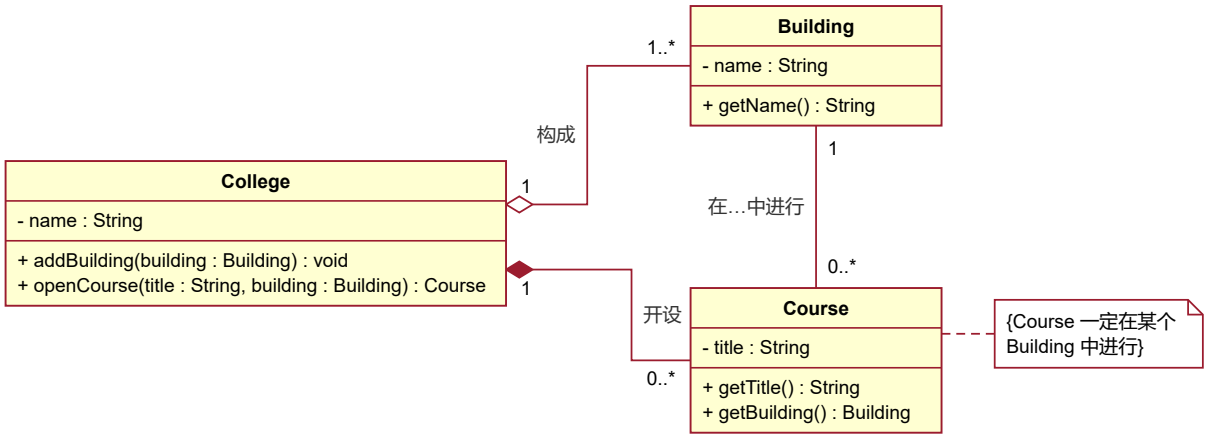



图3 A1-2 College / Building / Course 类图

**代码：**

Building.java：

```
package a1_2;

/** 建筑。由外部创建后加入学院，学院撤销后建筑仍然存在（聚合）。 */
public class Building {
```

```
    private final String name;

    public Building(String name) {
        this.name = name;
    }

    public String getName() {
        return name;
    }
}
```

Course.java：

```
package a1_2;

import java.util.Objects;

/** 课程。只能由 College 创建（组合），并且必须指定上课的建筑。 */
public class Course {
    private final String title;
    private final Building building;

    /** 包内可见：外部只能通过 College.openCourse 创建课程。 */
    Course(String title, Building building) {
        this.title = title;
        // 约束：Course 一定在某个 Building 中进行
        this.building = Objects.requireNonNull(building, "Course 必须在某个 Building 中进行");
    }

    public String getTitle() {
        return title;
    }

    public Building getBuilding() {
        return building;
    }
}
```

College.java：

```
package a1_2;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

/** 学院：聚合若干 Building，组合若干 Course。 */
public class College {
    private final String name;
    /** 聚合：建筑由外部创建并传入，可以独立于学院存在。 */
    private final List<Building> buildings = new ArrayList<>();
    /** 组合：课程由学院创建，随学院一起销毁。 */
    private final List<Course> courses = new ArrayList<>();

    public College(String name) {
        this.name = name;
    }

    public void addBuilding(Building building) {
```

```
        if (!buildings.contains(building)) {
            buildings.add(building);
        }
    }

    /** 开设课程。上课地点必须是本学院的建筑。 */
    public Course openCourse(String title, Building building) {
        if (building == null) {
            throw new IllegalArgumentException("Course 必须在某个 Building 中进行");
        }
        if (!buildings.contains(building)) {
            throw new IllegalArgumentException("课程只能安排在本学院的建筑中：" + building.getName
());
        }
        Course course = new Course(title, building);
        courses.add(course);
        return course;
    }

    public String getName() {
        return name;
    }

    public List<Building> getBuildings() {
        return Collections.unmodifiableList(buildings);
    }

    public List<Course> getCourses() {
        return Collections.unmodifiableList(courses);
    }
}
```

**运行结果：**

演示程序为学院加入两栋建筑并开设三门课程；在不属于本学院的建筑中开课、或者不指定建筑开课时，openCourse 都会抛出异常，结果如图4 所示。

```
$ java -Dstdout.encoding=UTF-8 -cp out a1_2.Demo
=== A1-2 College / Building / Course ===
计算机与信息学院 的建筑：
  J1 教学楼
  实验楼
计算机与信息学院 的课程：
  软件工程 @ J1 教学楼
  数据结构 @ J1 教学楼
  软件工程实验 @ 实验楼
在非本学院建筑开课：课程只能安排在本学院的建筑中：体育馆
不指定建筑开课：Course 必须在某个 Building 中进行
```

图4 A1-2 运行结果

## 3. A1-3 学院 School、系、教师、课程与学生

**描述：**

一个学院 School 由 1 到多个系 Department 构成；每个系至少拥有 1 位指导教师 Instructor，并提供至少 1 门课程 Subject；每个指导教师至少属于某个系且教授 1-3 门课程；学院拥有 0 到多名学生，学

生 Student 参加 1-5 门课程学习。请使用 UML 描述以上各类间的关系。

**建模分析：**

题目中的每句话对应类图中的一条关系，如表3 所示。题目只给出了部分关系一端的多重性，另一端按常识补全，并在说明中注明。

表3 A1-3 题目描述与类图关系的对应

| 题目描述 | 关系 | 多重性 | 说明 |
|---|---|---|---|
| 学院 School 由 1 到多个系 Department 构成 | School ◆— Department<br>组合 | School 1<br>Department 1..* | 系不能脱离学院单独存在 |
| 每个系至少拥有 1 位指导教师；每个指导教师至少属于某个系 | Department ◇— Instructor<br>聚合 | Department 1..*<br>Instructor 1..* | 一位教师可以同时属于多个系，部分被共享，所以是聚合而不是组合 |
| 每个系提供至少 1 门课程 | Department — Subject<br>关联“开设” | Department 1..*<br>Subject 1..* | 系一端为补全：一门课程至少由 1 个系开设，允许几个系合开 |
| 每个指导教师教授 1-3 门课程 | Instructor — Subject<br>关联“讲授” | Instructor 1..*<br>Subject 1..3 | 教师一端为补全：每门课程至少有 1 位教师讲授 |
| 学院拥有 0 到多名学生 | School ◇— Student<br>聚合 | School 1<br>Student 0..* | 学生属于 1 个学院，但学生对象可以独立于学院存在 |
| 学生参加 1-5 门课程学习 | Student — Subject<br>关联“参加” | Student 0..*<br>Subject 1..5 | 学生一端为补全：一门课程可以暂时没有学生选 |

代码中多重性的落实方式：

（1）上限：`Instructor.teach` 和 `Student.attend` 在已达到 MAX_SUBJECTS（分别为 3 和 5，类图中为带下划线的静态属性）时抛出异常；

（2）下限：创建对象的那一刻往往还无法满足（例如系刚创建时还没有教师），因此由 `School.checkMultiplicity()` 在模型建好之后统一检查，返回不满足的条目；

（3）系与教师是多对多的双向关联：`Department.addInstructor` 在加入教师的同时调用 `Instructor.joinDepartment`，保证两端一致；joinDepartment 是包内可见的（~），外部代码不能只修改一端。

**类图：**

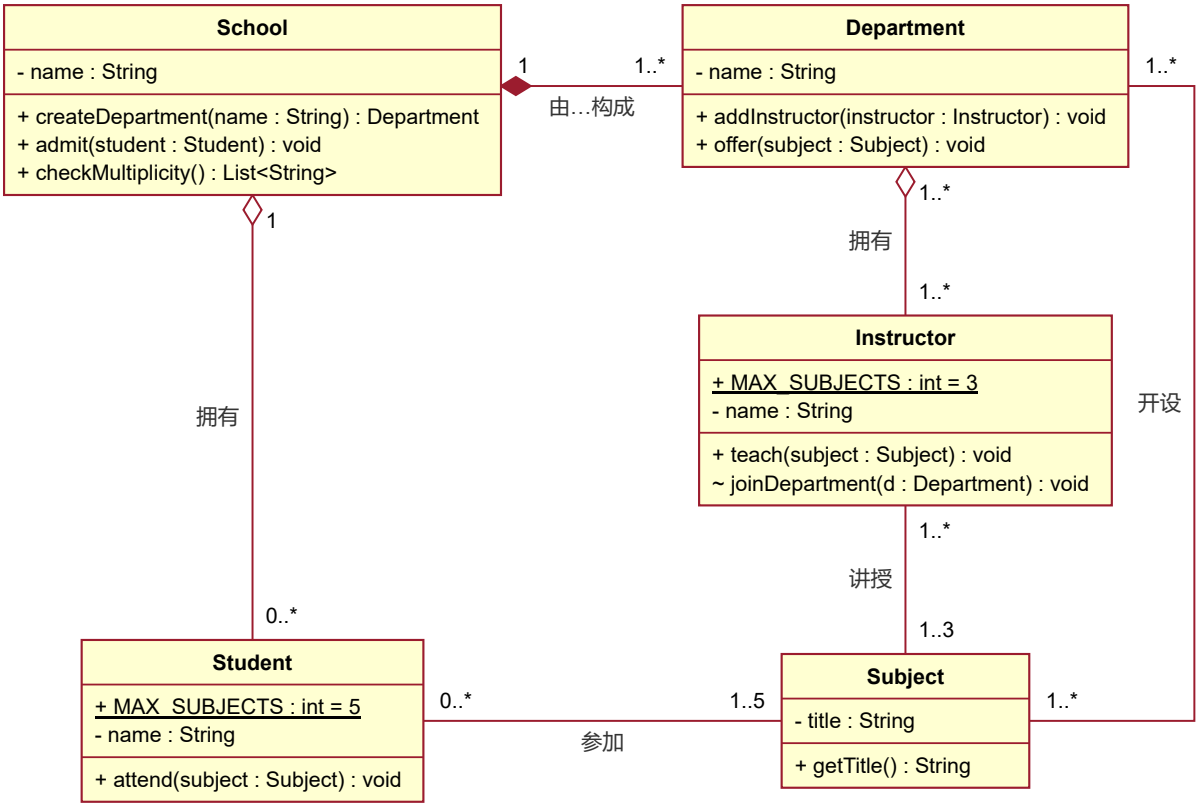



图5 A1-3 School / Department / Instructor / Subject / Student 类图

**代码：**

School.java：

```
package a1_3;

import java.util.ArrayList;
import java.util.Collections;
import java.util.LinkedHashSet;
import java.util.List;
import java.util.Set;

/** 学院：由 1 到多个系构成（组合），拥有 0 到多名学生。 */
public class School {
    private final String name;
    private final List<Department> departments = new ArrayList<>();
    private final List<Student> students = new ArrayList<>();

    public School(String name) {
        this.name = name;
    }

    /** 组合：系由学院创建，不能脱离学院单独存在。 */
    public Department createDepartment(String departmentName) {
        Department department = new Department(departmentName);
        departments.add(department);
        return department;
    }
```

```
public void admit(Student student) {
    if (!students.contains(student)) {
        students.add(student);
    }
}

/**
 * 检查各关联多重性的下限（上限在 teach/attend 时已检查），
 * 返回不满足的条目，全部满足时返回空列表。
 */
public List<String> checkMultiplicity() {
    List<String> problems = new ArrayList<>();
    if (departments.isEmpty()) {
        problems.add(name + " 至少要有 1 个系");
    }
    Set<Instructor> instructors = new LinkedHashSet<>();
    Set<Subject> offered = new LinkedHashSet<>();
    for (Department d : departments) {
        if (d.getInstructors().isEmpty()) {
            problems.add(d.getName() + " 至少要有 1 位指导教师");
        }
        if (d.getSubjects().isEmpty()) {
            problems.add(d.getName() + " 至少要开设 1 门课程");
        }
        instructors.addAll(d.getInstructors());
        offered.addAll(d.getSubjects());
    }
    for (Instructor i : instructors) {
        if (i.getSubjects().isEmpty()) {
            problems.add(i.getName() + " 至少要教授 1 门课程");
        }
        for (Subject s : i.getSubjects()) {
            if (!offered.contains(s)) {
                problems.add(i.getName() + " 教授的 " + s.getTitle() + " 没有由任何系开设");
            }
        }
    }
    for (Student st : students) {
        if (st.getSubjects().isEmpty()) {
            problems.add(st.getName() + " 至少要参加 1 门课程");
        }
        for (Subject s : st.getSubjects()) {
            if (!offered.contains(s)) {
                problems.add(st.getName() + " 参加的 " + s.getTitle() + " 没有由任何系开设");
            }
        }
    }
    for (Subject s : offered) {
        boolean taught = instructors.stream().anyMatch(i -> i.getSubjects().contains(s));
        if (!taught) {
            problems.add(s.getTitle() + " 至少要有 1 位教师讲授");
        }
    }
    return problems;
}

public String getName() {
    return name;
}
```

```
    public List<Department> getDepartments() {
        return Collections.unmodifiableList(departments);
    }

    public List<Student> getStudents() {
        return Collections.unmodifiableList(students);
    }
}
```

Department.java：

```
package a1_3;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

/** 系：至少拥有 1 位指导教师，至少开设 1 门课程。 */
public class Department {
    private final String name;
    private final List<Instructor> instructors = new ArrayList<>();
    private final List<Subject> subjects = new ArrayList<>();

    /** 包内可见：系只能由 School.createDepartment 创建（组合）。 */
    Department(String name) {
        this.name = name;
    }

    /** 一位教师可以同时属于多个系，关联两端同时更新。 */
    public void addInstructor(Instructor instructor) {
        if (!instructors.contains(instructor)) {
            instructors.add(instructor);
            instructor.joinDepartment(this);
        }
    }

    public void offer(Subject subject) {
        if (!subjects.contains(subject)) {
            subjects.add(subject);
        }
    }

    public String getName() {
        return name;
    }

    public List<Instructor> getInstructors() {
        return Collections.unmodifiableList(instructors);
    }

    public List<Subject> getSubjects() {
        return Collections.unmodifiableList(subjects);
    }
}
```

Instructor.java：

```
package a1_3;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

/** 指导教师：至少属于 1 个系，教授 1~3 门课程。 */
public class Instructor {
    public static final int MAX_SUBJECTS = 3;

    private final String name;
    private final List<Department> departments = new ArrayList<>();
    private final List<Subject> subjects = new ArrayList<>();

    public Instructor(String name) {
        this.name = name;
    }

    /** 由 Department.addInstructor 调用，维护双向关联。 */
    void joinDepartment(Department department) {
        if (!departments.contains(department)) {
            departments.add(department);
        }
    }

    public void teach(Subject subject) {
        if (subjects.contains(subject)) {
            return;
        }
        if (subjects.size() >= MAX_SUBJECTS) {
            throw new IllegalStateException(name + " 已教授 " + MAX_SUBJECTS + " 门课程，不能再增
加");
        }
        subjects.add(subject);
    }

    public String getName() {
        return name;
    }

    public List<Department> getDepartments() {
        return Collections.unmodifiableList(departments);
    }

    public List<Subject> getSubjects() {
        return Collections.unmodifiableList(subjects);
    }
}
```

Student.java：

```
package a1_3;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
```

```
/** 学生：参加 1~5 门课程。 */
public class Student {
    public static final int MAX_SUBJECTS = 5;

    private final String name;
    private final List<Subject> subjects = new ArrayList<>();

    public Student(String name) {
        this.name = name;
    }

    public void attend(Subject subject) {
        if (subjects.contains(subject)) {
            return;
        }
        if (subjects.size() >= MAX_SUBJECTS) {
            throw new IllegalStateException(name + " 已参加 " + MAX_SUBJECTS + " 门课程，不能再增
加");
        }
        subjects.add(subject);
    }

    public String getName() {
        return name;
    }

    public List<Subject> getSubjects() {
        return Collections.unmodifiableList(subjects);
    }
}
```

Subject.java：

```
package a1_3;

/** 课程。 */
public class Subject {
    private final String title;

    public Subject(String title) {
        this.title = title;
    }

    public String getTitle() {
        return title;
    }
}
```

**运行结果：**

演示程序建立了 2 个系、3 位教师（王老师同时属于两个系）、6 门课程和 2 名学生，多重性检查全部满足；随后让李老师教第 4 门课、学生甲选第 6 门课，都被拒绝；最后加入一名未选课的学生和一个空的系，checkMultiplicity 列出了 3 条不满足的下限，结果如图6 所示。

```
$ java -Dstdout.encoding=UTF-8 -cp out a1_3.Demo
=== A1-3 School / Department / Instructor / Subject / Student ===
计算机与信息学院
  计算机科学系  开设：[数据结构，操作系统，计算机网络]  教师：[王老师，张老师]
  软件工程系  开设：[软件工程，数据库原理，Java程序设计]  教师：[李老师，王老师]
  王老师  所属系：[计算机科学系，软件工程系]  讲授：[数据结构，操作系统]
  张老师  所属系：[计算机科学系]  讲授：[计算机网络]
  李老师  所属系：[软件工程系]  讲授：[软件工程，数据库原理，Java程序设计]
  学生甲  参加：[数据结构，软件工程，数据库原理]
  学生乙  参加：[操作系统，计算机网络]
多重性检查：全部满足
李老师教第 4 门课：李老师 已教授 3 门课程，不能再增加
学生甲选第 6 门课：学生甲 已参加 5 门课程，不能再增加
新增未选课的学生丙和空的人工智能系后：
多重性检查：
  × 人工智能系 至少要有 1 位指导教师
  × 人工智能系 至少要开设 1 门课程
  × 学生丙 至少要参加 1 门课程
```

图6 A1-3 运行结果

## 4. A1-4 网上购物系统

**描述：**

客户（Customer）具有四个属性：姓名（name）、联系电话（contact）、邮寄地址（deliveryAddress）和是否激活状态（active）；订单（Order）具有两个属性：创建日期（createDate）、订单状态（status），订单状态包括 CREATE、PAID、SHIPPING、DELIVERED、CANCEL 五种状态；订单详情（OrderDetail）具有一个属性：数量（quantity），并具有 calculateSubTotal()、calculateWeight() 方法；商品项（Product）具有名称（title）、重量（weight）和描述（description），并具有 getPriceForQuantity() 方法和 getWeight() 方法；支付方式（Payment）具有金额（amount）属性；支付方式支持以下五种：信用卡 Credit，具有卡号（number）、类型（type）和过期日期（expireDate）；现金（Cash），具有支付金额（cashTendered）属性；在线转账（WireTransfer），具有银行 ID（bankID）、银行名称 bankName 属性；支付宝（AliPay）具有卡号 number 属性；微信支付（WeixinPay），具有卡号属性。请根据以上简化的网上购物系统的关键类描述说明，创建类模型，并定义类之间的关系包括多重性（Multiplicity）。

**建模分析：**

系统中的类及其成员如表4 所示，类之间的关系如表5 所示。

表4 A1-4 类及其成员

| 类 | 属性（均为私有） | 操作（均为公有） | 说明 |
| --- | --- | --- | --- |
| Customer | name, contact,<br>deliveryAddress : String<br>active : boolean | placeOrder(createDate : LocalDate)<br>: Order | 未激活的客户不能下单 |
| Order | createDate : LocalDate<br>status : OrderStatus | addDetail, addPayment, setStatus | 新建订单的状态为 CREATE |
| OrderStatus | CREATE, PAID, SHIPPING,<br>DELIVERED, CANCEL | — | 枚举 |

| | | | |
|---|---|---|---|
| OrderDetail | quantity : int | calculateSubTotal() : double<br>calculateWeight() : double | 小计金额由 Product.getPriceForQuantity 计算 |
| Product | title, description : String<br>weight, price : double | getPriceForQuantity(quantity : int) : double<br>getWeight() : double | price 为补充的属性，见下文 |
| Payment | amount : double | getAmount() : double | 抽象类 |
| Credit | number, type : String<br>expireDate : LocalDate | | 信用卡 |
| Cash | cashTendered : double | | 现金 |
| WireTransfer | bankID, bankName : String | | 在线转账 |
| AliPay | number : String | | 支付宝 |
| WeixinPay | number : String | | 微信支付 |

表5 A1-4 类之间的关系

| 关系 | 类型 | 多重性 | 说明 |
|---|---|---|---|
| Customer — Order | 关联“下单” | Customer 1<br>Order 0..* | 一个客户可以有 0 到多个订单，每个订单属于 1 个客户 |
| Order — OrderDetail | 组合（实心菱形在 Order 端） | Order 1<br>OrderDetail 1..* | 订单至少有 1 条详情，详情随订单创建和删除 |
| OrderDetail — Product | 关联“对应” | OrderDetail 0..*<br>Product 1 | 每条详情对应 1 种商品，一种商品可以出现在多条详情中 |
| Order — Payment | 关联“支付” | Order 1<br>Payment 0..* | 订单处于 CREATE 或 CANCEL 状态时可以没有支付；也允许分多笔、用不同方式支付 |
| Credit / Cash / WireTransfer / AliPay / WeixinPay — Payment | 泛化 | — | 五种支付方式继承 Payment，共同的 amount 放在父类 |
| Order → OrderStatus | 依赖 | — | status 属性的类型是枚举 OrderStatus |

设计说明：

（1）Payment 设为抽象类：每笔支付一定是五种方式之一，不应直接创建 Payment 对象；金额 amount 是共同属性，放在父类中，子类只增加各自的属性。

（2）题目中的商品项只有名称、重量和描述，没有单价，无法实现 getPriceForQuantity()，因此给 Product 增加了私有属性 price（单价）。

（3）订单状态用枚举 OrderStatus 表示，取值只能是题目给出的五种，比用字符串或整数更安全。

（4）Order 和 OrderDetail 的构造方法都是包内可见的：订单只能由 Customer.placeOrder 创建，订单详情只能由 Order.addDetail 创建，与类图中的关联和组合关系一致。

**类图：**

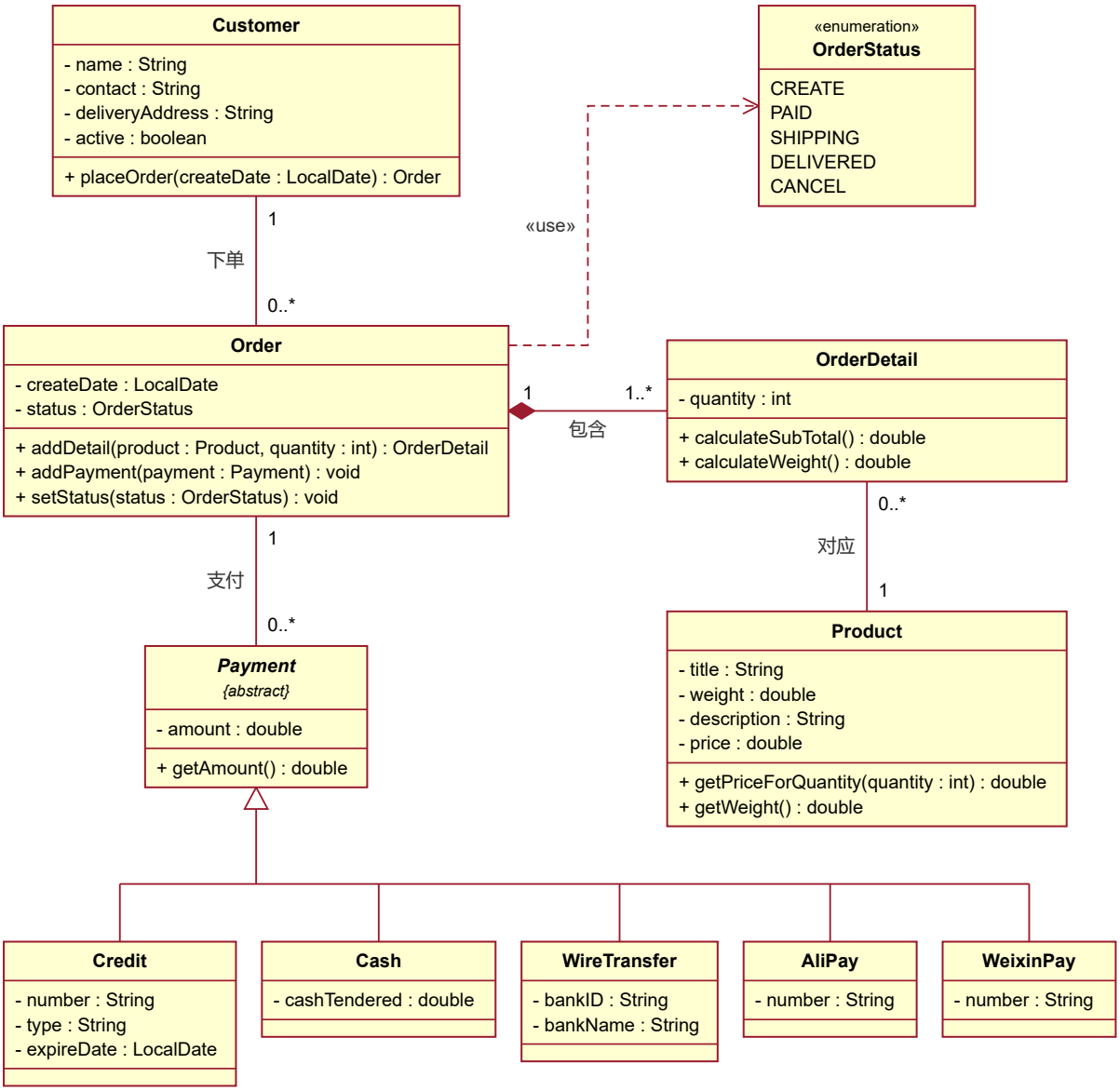



图7 A1-4 网上购物系统类图

**代码：**

OrderStatus.java：

```
package a1_4;

/** 订单状态。 */
public enum OrderStatus {
    CREATE, PAID, SHIPPING, DELIVERED, CANCEL
}
```

Customer.java：

```
package a1_4;

import java.time.LocalDate;
```

```
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

/** 客户：一个客户可以有 0 到多个订单。 */
public class Customer {
    private String name;
    private String contact;
    private String deliveryAddress;
    private boolean active;
    private final List<Order> orders = new ArrayList<>();

    public Customer(String name, String contact, String deliveryAddress, boolean active) {
        this.name = name;
        this.contact = contact;
        this.deliveryAddress = deliveryAddress;
        this.active = active;
    }

    /** 下单。新订单的状态为 CREATE；未激活的客户不能下单。 */
    public Order placeOrder(LocalDate createDate) {
        if (!active) {
            throw new IllegalStateException(name + " 未激活，不能下单");
        }
        Order order = new Order(this, createDate);
        orders.add(order);
        return order;
    }

    public String getName() {
        return name;
    }

    public String getContact() {
        return contact;
    }

    public String getDeliveryAddress() {
        return deliveryAddress;
    }

    public boolean isActive() {
        return active;
    }

    public void setActive(boolean active) {
        this.active = active;
    }

    public List<Order> getOrders() {
        return Collections.unmodifiableList(orders);
    }
}
```

Order.java：

```
package a1_4;

import java.time.LocalDate;
```

```
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

/** 订单：属于 1 个客户，由 1 到多条订单详情组成，可以有 0 到多笔支付。 */
public class Order {
    private final LocalDate createDate;
    private OrderStatus status = OrderStatus.CREATE;
    private final Customer customer;
    private final List<OrderDetail> details = new ArrayList<>();
    private final List<Payment> payments = new ArrayList<>();

    /** 包内可见：订单只能通过 Customer.placeOrder 创建。 */
    Order(Customer customer, LocalDate createDate) {
        this.customer = customer;
        this.createDate = createDate;
    }

    /** 组合：订单详情由订单创建，随订单一起删除。 */
    public OrderDetail addDetail(Product product, int quantity) {
        OrderDetail detail = new OrderDetail(product, quantity);
        details.add(detail);
        return detail;
    }

    /** 一个订单可以分多笔、用不同方式支付。 */
    public void addPayment(Payment payment) {
        payments.add(payment);
    }

    public LocalDate getCreateDate() {
        return createDate;
    }

    public OrderStatus getStatus() {
        return status;
    }

    public void setStatus(OrderStatus status) {
        this.status = status;
    }

    public Customer getCustomer() {
        return customer;
    }

    public List<OrderDetail> getDetails() {
        return Collections.unmodifiableList(details);
    }

    public List<Payment> getPayments() {
        return Collections.unmodifiableList(payments);
    }
}
```

OrderDetail.java：

```
package a1_4;
```

```
import java.util.Objects;

/** 订单详情：某个商品及其购买数量。 */
public class OrderDetail {
    private final int quantity;
    private final Product product;

    /** 包内可见：只能由 Order.addDetail 创建。 */
    OrderDetail(Product product, int quantity) {
        if (quantity <= 0) {
            throw new IllegalArgumentException("数量必须大于 0");
        }
        this.product = Objects.requireNonNull(product, "订单详情必须对应一个商品");
        this.quantity = quantity;
    }

    /** 小计金额 = 该商品在此数量下的价格。 */
    public double calculateSubTotal() {
        return product.getPriceForQuantity(quantity);
    }

    /** 小计重量 = 单件重量 × 数量。 */
    public double calculateWeight() {
        return product.getWeight() * quantity;
    }

    public int getQuantity() {
        return quantity;
    }

    public Product getProduct() {
        return product;
    }
}
```

Product.java：

```
package a1_4;

/** 商品项。 */
public class Product {
    private final String title;
    /** 单件重量（kg）。 */
    private final double weight;
    private final String description;
    /** 单价（元）。题目没有给出，为实现 getPriceForQuantity 而增加。 */
    private final double price;

    public Product(String title, double weight, String description, double price) {
        this.title = title;
        this.weight = weight;
        this.description = description;
        this.price = price;
    }

    public double getPriceForQuantity(int quantity) {
        return price * quantity;
    }
```

```
    public double getWeight() {
        return weight;
    }

    public String getTitle() {
        return title;
    }

    public String getDescription() {
        return description;
    }
}
```

Payment.java：

```
package a1_4;

/** 支付方式（抽象类），具体方式见各子类。 */
public abstract class Payment {
    private final double amount;

    protected Payment(double amount) {
        if (amount <= 0) {
            throw new IllegalArgumentException("支付金额必须大于 0");
        }
        this.amount = amount;
    }

    public double getAmount() {
        return amount;
    }
}
```

Credit.java：

```
package a1_4;

import java.time.LocalDate;

/** 信用卡支付。 */
public class Credit extends Payment {
    private final String number;
    private final String type;
    private final LocalDate expireDate;

    public Credit(double amount, String number, String type, LocalDate expireDate) {
        super(amount);
        this.number = number;
        this.type = type;
        this.expireDate = expireDate;
    }

    public String getNumber() {
        return number;
    }

    public String getType() {
        return type;
    }
```

```
    public LocalDate getExpireDate() {
        return expireDate;
    }

    @Override
    public String toString() {
        return String.format("信用卡 %s %s（有效期至 %s）  %.2f 元", type, number, expireDate, ge
tAmount());
    }
}
```

Cash.java：

```
package a1_4;

/** 现金支付（如货到付款）。 */
public class Cash extends Payment {
    private final double cashTendered;

    public Cash(double amount, double cashTendered) {
        super(amount);
        if (cashTendered < amount) {
            throw new IllegalArgumentException("支付的现金不能少于应付金额");
        }
        this.cashTendered = cashTendered;
    }

    public double getCashTendered() {
        return cashTendered;
    }

    @Override
    public String toString() {
        return String.format("现金（实收 %.2f 元，找零 %.2f 元）  %.2f 元",
                cashTendered, cashTendered - getAmount(), getAmount());
    }
}
```

WireTransfer.java：

```
package a1_4;

/** 在线转账。 */
public class WireTransfer extends Payment {
    private final String bankID;
    private final String bankName;

    public WireTransfer(double amount, String bankID, String bankName) {
        super(amount);
        this.bankID = bankID;
        this.bankName = bankName;
    }

    public String getBankID() {
        return bankID;
    }

    public String getBankName() {
```

```
        return bankName;
    }

    @Override
    public String toString() {
        return String.format("在线转账 %s（%s）  %.2f 元", bankName, bankID, getAmount());
    }
}
```

AliPay.java：

```
package a1_4;

/** 支付宝支付。 */
public class AliPay extends Payment {
    private final String number;

    public AliPay(double amount, String number) {
        super(amount);
        this.number = number;
    }

    public String getNumber() {
        return number;
    }

    @Override
    public String toString() {
        return String.format("支付宝 %s  %.2f 元", number, getAmount());
    }
}
```

WeixinPay.java：

```
package a1_4;

/** 微信支付。 */
public class WeixinPay extends Payment {
    private final String number;

    public WeixinPay(double amount, String number) {
        super(amount);
        this.number = number;
    }

    public String getNumber() {
        return number;
    }

    @Override
    public String toString() {
        return String.format("微信支付 %s  %.2f 元", number, getAmount());
    }
}
```

**运行结果：**

演示程序为同一个客户创建了 4 个订单：订单 1 用支付宝和微信分两笔付清后发货、送达；订单 2 用信用卡和在线转账支付；订单 3 货到付款，用现金支付并找零；订单 4 下单后未支付就取消，没有任何支付记录，这也是 Order 一端到 Payment 的多重性取 0..* 的原因。最后把客户设为未激活，再下单时被拒绝。结果如图8 所示。

```
$ java -Dstdout.encoding=UTF-8 -cp out a1_4.Demo
=== A1-4 网上购物系统 ===
客户：张三，电话 138-0000-0000，地址 某市某区学府路 1 号，激活 true
订单 1  创建日期 2026-10-14  状态 DELIVERED
    机械键盘 × 1  小计 299.00 元  重量 1.20 kg
    无线鼠标 × 2  小计 178.00 元  重量 0.30 kg
    《软件工程》教材 × 1  小计 59.00 元  重量 0.80 kg
    合计 536.00 元，总重 2.30 kg
    支付：支付宝 zhangsan@example.com  300.00 元
    支付：微信支付 wxid_zhangsan  236.00 元
订单 2  创建日期 2026-10-15  状态 PAID
    机械键盘 × 2  小计 598.00 元  重量 2.40 kg
    合计 598.00 元，总重 2.40 kg
    支付：信用卡 VISA 6222-0000-0000-0001（有效期至 2028-12-31）  298.00 元
    支付：在线转账 中国工商银行（ICBC-0001）  300.00 元
订单 3  创建日期 2026-10-16  状态 PAID
    《软件工程》教材 × 2  小计 118.00 元  重量 1.60 kg
    合计 118.00 元，总重 1.60 kg
    支付：现金（实收 150.00 元，找零 32.00 元）  118.00 元
订单 4  创建日期 2026-10-16  状态 CANCEL
    无线鼠标 × 1  小计 89.00 元  重量 0.15 kg
    合计 89.00 元，总重 0.15 kg
    支付：无
未激活客户下单：张三 未激活，不能下单
```

图8 A1-4 运行结果

## 5. 代码提交

代码和类图提交到 GitHub 仓库的 `lab3-oo-modeling` 目录，提交记录为 `13d9b4d Add lab 3 OO class models (A1-1 to A1-4) with UML diagrams`，本地 master 与 origin/master 一致，如图9 所示。

```
$ git log --oneline -4 && git status -sb
13d9b4d Add lab 3 OO class models (A1-1 to A1-4) with UML diagrams
f34e530 docs: add reading reflection and summary to lab report 1
686efdf docs: add lab report 1
2924866 test: add unit tests for HJ212Parser
## master...origin/master
```

图9 提交记录

# 五、实验结论

**心得体会：**

通过本次实验，练习了用 UML 类图描述类的属性、操作和可见性，以及类之间的关联、聚合、组合、泛化、依赖和多重性，并用 Java 把四个模型实现出来。体会比较深的有三点：

一是聚合和组合的区别不只是“整体与部分”，关键在于部分能否被共享、能否独立于整体存在。例如学院和系是组合，而系和教师是聚合，因为一位教师可以同时属于多个系。

二是多重性写进代码后就成了需要检查的约束：上限可以在添加对象时直接检查，下限往往要等模型建好之后再统一校验。

三是题目描述常常只给出关系一端的多重性，另一端需要结合常识补全并说明理由；题目中缺少的信息（如商品单价）也要在设计时补上，否则方法无法实现。

# 六、仓库地址

https://github.com/advance-cmd/Course---Software-Engineering

本实验代码：https://github.com/advance-cmd/Course---Software-Engineering/tree/master/lab3-oo-modeling